



MADHU

Taskbar Kitten · Master Reference & Prompt

A personal Windows taskbar companion cat and private keepsake journal • Built with affection, documented with care • *Everything you need to understand, maintain, and extend this project*

Project Type:	Personal Windows App (Python, Tkinter, Pillow)
For:	Gayathree — a personal gift
Dual Purpose:	Virtual Pet + Private Journal
Generated:	September 19, 2026

Table of Contents

1. Identity
2. Who It Is For
3. Feature List (Everything)
4. Architecture & Files
5. Menus
6. Memory & Persistence
7. The Journal
8. Animation System
9. Walking & Taskbar Placement
10. Sound
11. App Glue, Tray & Threading
12. Activity Monitor
13. Cooldowns & Behavior Timing
14. Testing
15. Build & Release Pipeline
16. Git & Repository Hygiene
17. Known Limitations
18. Gotchas for Future Edits
19. The Master Prompt

1. Identity

Madhu is a tiny 56×56 pixel cat that lives on the Windows taskbar. She is two things at once:

1. **A Virtual Pet** — she walks, sleeps, gets petted, wishes luck, celebrates milestones, and reacts to how long you have been away.
2. **A Private Keepsake Journal Keeper** — "Gayathree's Journal", a book that "Madhu keeps for you", with daily pages, moods, ratings, tags, photos, rich-text formatting, statistics, a password lock, and export.

Naming

On first run a dialog asks "what will you call her?" (default suggestion "**Madhu**"). That name appears everywhere: the root window title, the right-click menu item *Pet* <*n*> <3, and the tray. Stored in memory under name.

Personality & Voice

Affectionate, low-key, playful, reassuring, warm; small hearts <3, short sentences, all text in English.

2. Who It Is For

Gayathree — the person Madhu was built for. An architecture / studio / 3D-modeling student, which is why the encouragement ("luck") lines are architecture- and model-building-themed (e.g. *"Architecture is patience — your lines will find their place"*, *"One more extrude and you will see it"*).

Single user, personal gift, Windows-only: uses Win32 APIs (taskbar rect, named mutex, MessageBox, winsound). It will never need to be macOS/Linux.

Not a product, not public. Treat voice/spelling changes as personal taste, not bugs to "normalize".

3. Feature List (Everything)

3.1 Pet Behaviors

Feature	Where / Trigger	Notes
Sits on taskbar	startup	placement <i><i>above</i></i> (floating) or <i><i>overlay</i></i> (sits ON the b
Idle animation	always	<i><i>frame_0..7.png</i></i> idle frames
Sleeping	idle timeout	slow-breathing sleep frames; night (22:00-06:00) sleeps right awa
Nap easter-egg	during sleep, rare	occasional stretch/blink out of sleep, then back; 90 s cooldown
Walking	auto-bored + menu	clamped inside the taskbar lane; pauses for pet/luck
Petting	left-click, menu	pet animation + counter + optional meow + occasional heart
Mood on startup	app start	night = sleeps; gone >= 3 days = big greeting; 1 day = small
Long-absence check	every ~30 min	bubble when away longer than threshold (default 2 h)
Duck mode (coding)	menu + auto via activity	"duck = don't bug me": no walking, sits/sleeps
Don't sleep 30m	menu	blocks sleeping for 30 min; menu label counts down

3.2 Talk & Bubbles

Speech bubble above the kitten. Signature: `_show_bubble(text, ms=4000, priority=0)`. Priority 1 preempts/merges. Messages currently fired include name dialog, fruit/wake/long absence notifications, milestones, 11:11 wishes, seasonal messages, and journal reminders. Text lives inline in `pet_window.py` (LUCK_MESSAGES, milestone strings, greetings).

3.3 Luck & Encouragement

Wish me luck — menu + tray. Rotates sequentially through LUCK_MESSAGES (60+ architecture/studio lines); index stored in memory. Bubble priority 1; walk pauses during it.

3.4 Milestones (Persisted Once, Never Repeated)

Milestone	Trigger
<code><code>pet1</code></code>	first pet ever — "First pet! <i><n></i> loves you"
<code><code>pet10</code></code>	10 pets — "10 pets already? Lucky <i><n></i> "
<code><code>pet100</code></code>	100 pets — "100 pets! <i><n></i> is so loved"
<code><code>pet500</code></code>	500 pets — "500 pets! You're <i><n></i> 's favorite human"
<code><code>week1</code></code>	7 days since birth — "1 week together!"
<code><code>month1</code></code>	30 days since birth — "1 month with <i><n></i> !"

All newly-fired milestones in one tick merge into ONE bubble (`" ; ".join`), one meow, one size "pop" burst.

3.5 The Journal (The "Book") — Internals in Section 7

Password gate, then a book-styled window (parchment, spine, two pages):

- Daily pages (one entry per date), auto-create, backdate, delete
- Mood: one of 6 emojis (buttons). Rating: 1–5 hearts (tap to unset)
- Tags: 6 fixed — love, us, study, thoughts, dreams, today
- Rich text: bold / italic / underline / color / link / heading / bullet / todo checkbox, re-applied on load
- Photos: attach images, stored under `journal_photos/`, inline thumbnails
- Word count, debounced auto-save, manual "Save for Madhu"
- Search filters the day list (never steals selection while searching)
- Calendar & insights: streak, top tags, mood/rating trends; click a day to jump
- Dark mode toggle (per-session via `_journal_dark_cur`)
- Export text (.txt) and PDF (.pdf) — ALL or today-only (multi-page PDF)
- Password lock (Fernet + scrypt) and reminder (see Section 3.2)

3.6 Journal Reminder

Default 21:00; change via the reminder button (HH:MM, 24 h). Stored in memory `journal_hour/journal_min`. Fires for the WHOLE target minute, once per day (latched by `date_hour_minute` slot).

3.7 Our Story

Small read-only scrapbook: days together, pet counts, milestones, longest time apart. Not a separate journal.

3.8 Tray (pystray)

Icon = `frame_4.png`. Menu with dynamic labels: Placement: Above, Placement: Overlay (ON taskbar), separator, Pet <n> <3, Wish me luck, Our story, Mute/Unmute (label updates live via `icon.update_menu()`), Quit.

3.9 System Integration

- **Single instance** via Win32 mutex; second launch shows a MessageBox "Madhu is already running..." and exits.
- **Auto-open journal:** env `KITTY_OPEN_JOURNAL=1` or arg `--open-journal`.

4. Architecture & Files

The project lives at D:\KITTY\ and is organized as a single-entry-point application with modular file structure:

File	Purpose
MASTER.md	This documentation — the single source of truth
src/app.py	Entry point: mutex, memory, tray, monitor, mainloop
src/pet_window.py	PetWindow class: ALL UI, behaviors, journal (~4700 lines)
src/animotor.py	SpriteAnimator: state machine / frame selection
src/taskbar.py	Win32 taskbar geometry helpers
src/activity.py	ActivityMonitor: CPU/process polling (1 s interval)
src/memory.py	JSON memory manager + migration logic
assets/	Sprites, sounds, journal data (dev) & photos
installer/	Inno Setup config for <code>KittySetup.exe</code>

Key class: PetWindow in pet_window.py is the main window and behavior hub. All UI elements, animations, journal logic, speech bubbles, and milestone tracking happen here. The file is large (~4700 lines) but is the authoritative place to find how Madhu works.

5. Menus & UI Layout

Right-click menu built into the main window (via Tkinter's popup). Menu indices COUNT SEPARATORS (this is load-bearing for entryconfig() calls). Index 4 = "Pet <name> <3", index 8 = "Don't sleep 30m", index 9 = "Mute/Unmute". See Section 18 (Gotchas) for menu index warnings.

6. Memory & Persistence

Runtime state: kitten_memory.json + .bak backup. Stores: name, pet count, mood latch, luck message index, don't-sleep timer, placement mode, journal reminder time, seasonal latches, and milestone flags.

Location: Dev builds → D:\KITTY\assets\. Frozen builds → %APPDATA%\TaskbarKitten\. One-time migration on first frozen run.

7. The Journal — Full Technical Breakdown

7.1 Data Format

Stored as journal.json (plaintext during dev, or journal.enc + journal.lock if password-protected). Structure: { "2025-01-15": { "mood": "■", "rating": 5, "tags": [...], "text": "...", "photos": [...] }, ... }

7.2 Password Lock (Fernet + scrypt)

- 6+ character password required. Encrypted via Fernet (symmetric) with a scrypt-derived key.
- 5-try throttle (3 s delay per failure).
- Atomic temp+replace writes + .bak backups.
- Fallback: .enc.bak if decrypt fails.

7.3 Photos

Stored on disk under journal_photos/ (dev: assets/, frozen: %APPDATA%). JSON stores relative paths + metadata. Inline thumbnails in the journal UI.

8. Animation System

SpriteAnimator in animator.py is a state machine: idle (frame 0–7), sleep (slow breathing), walking (frames), petting (pop), etc. Sprite frames live in assets/sprites/frame_N.png (56×56 PNG). The animator selects frames based on state and timing.

9. Walking & Taskbar Placement

Kitten walks left/right along the taskbar lane (clamped to avoid falling off edges). Auto-triggers after idle timeout ("bored"). Can be manually triggered via menu. Pauses during petting, luck wishes, and modals. Placement modes: above (floating) or overlay (sits ON the taskbar bar itself).

10. Sound

Single WAV file: assets/meow.wav. Throttled (1 s) to avoid spam. Toggleable via tray menu. Bundled via PyInstaller --add-data flag. Plays via winsound.PlaySound() on a non-blocking thread.

11. App Glue, Tray & Threading

Threading rule: Tkinter is main-thread-only. Cross-thread calls MUST use `pet.post_ui(...)` (drained every 300 ms). **Tray:** pystray icon with dynamic menu (labels update live via `icon.update_menu()`). **Activity monitor:** Runs on a daemon thread, polls CPU & foreground process every 1 s.

12. Activity Monitor

Detects when a coding editor is active (VS Code, JetBrains, Sublime, etc.) or CPU is high. Auto-engages **Duck mode** when detected ("duck = don't bug me"). In Duck mode: no walking, no luck wishes, Madhu just sits or sleeps. Manual override via menu "Duck mode (coding)". CPU threshold: 42% (tuned on the author's machine; YMMV).

13. Cooldowns & Behavior Timing

What	Timing
Sound throttle	1 s (<code><code>_last_sound_ts</code></code>)
Nap easter-egg	~90 s cooldown
Seasonal message	latched once per year key in memory
11:11 wish	fires when now == 11:11, once per minute green light
Journal reminder	whole target minute, once/day (slot latch)
11:11/seasonal/luck bubbles	priority 1 (preempts lower-priority bubbles)
UI queue drain	every 300 ms
Don't-sleep window	30 min, menu label counts down
Long-absence check	runs every ~30 min; threshold 2 h (configurable)

General rule: any deferred behavior uses `self.root.after(...)` with a `_ready_for_transition()` guard so it never fires mid-walk/pet/duck/modal.

14. Testing

Run from D:\KITTY (set PYTHONIOENCODING=utf-8 on Windows; suites build real widgets, so a desktop session is required):

```
python _test_journal.py - 16 checks (journal persistence/UI flows)
python _test_lock.py - 11 checks (password lock/encryption round-trips)
python %TEMP%\opencode\test_walkfix.py - 13 checks (walking lane/clamp behavior)
python %TEMP%\opencode\test_dontsleep.py - 23 checks (don't-sleep + duck interplay)
python -m py_compile src\app.py src\pet_window.py src\animator.py src\memory.py
src\activity.py src\taskbar.py
```

Menu index facts (4 = Pet, 8 = Don't sleep, 9 = Mute) are asserted by the suites — run them after any menu change.

15. Build & Release Pipeline

Step 1: PyInstaller build

```
python -m PyInstaller --noconfirm --windowed --onedir --name Kitty --add-data
"assets\sprites;assets\sprites" --add-data "assets\meow.wav;assets" --icon
"assets\sprites\frame_4.png" src\app.py
→ dist\Kitty\Kitty.exe (assets land in _internal\assets)
```

Step 2: Inno Setup installer

```
"C:\Program Files (x86)\Inno Setup 6\ISCC.exe" installer\kitty.iss
→ KittySetup.exe (root of the repo; TRACKED in git)
```

Step 3: Release

Commit + push; the raw GitHub URL serves the installer:
<https://raw.githubusercontent.com/ramsathwik2/something-something-/main/KittySetup.exe>

Step 4: Reinstall = Update mechanism

Download and run the setup. The app has no self-update; users download the new KittySetup.exe.

Smoke Checklist After Every Build (Frozen App):

- ✓ Single instance (second launch → MessageBox, no second kitten window)
- ✓ First run shows the name dialog; %APPDATA%\TaskbarKitten\ is populated
- ✓ dist\Kitty_internal\assets\meow.wav exists
- ✓ cryptography and PIL packages are bundled inside _internal

16. Git & Repository Hygiene

Repo: <https://github.com/ramsathwik2/something-something->, branch main

Staging: NEVER `git add -A`. Always stage targeted files.

Tracked: `src/*`, `build_installer.bat`, `installer/kitty.iss`, `KittySetup.exe`, `docs`

Gitignored: `assets/journal.json(.bak)`, `assets/journal.enc`, `assets/kitten_memory.json(.bak)`, `build/`, `dist/`, `*.spec`, `__pycache__`/

Commit style: `fix: ...`, `build: ...`, `chore: ...`

17. Known Limitations

- The password gate is an **in-app lock**, NOT OS-grade security. It keeps the journal safe from the casual reader.
- Dev and frozen builds keep **SEPARATE journals** (dev `assets/`, frozen `%APPDATA%`). One-time migration on first frozen run.
- During an in-place upgrade, an OLD running instance does not hold the NEW mutex, so a few seconds of two instances can occur.
- `winsound` is Windows-only (fine: the app is Windows-only).
- **Resolution/fonts:** hardcoded pixels and fixed dialog sizes; tuned on 1366×768 and may look off on other DPI.
- Two code copies of pet window logic (menu label + tray callables) can drift — tests catch menu drift.

18. Gotchas for Future Edits

- **Menu indices COUNT separators** (see Section 5); `entryconfig(4)/entryconfig(9)` are load-bearing.
- After any change to non-text/emoji-heavy behavior, run the six-line compile check + the four suites.
- `_ready_for_transition()` gates deferred `after()` lambdas; new behaviors must use it (`walk/pet/duck/modal`).
- `_save_journal` is skipped when `_journal_read_error` is `True`; do not turn that off or a corrupt journal gets repeatedly overwritten.
- Removing the journal lock writes plaintext DIRECTLY while the lock file still exists; do not call `_save_journal` there.
- Never reference hardcoded paths like `D:\KITTY`, `F:\`, or `C:\Users\...` in code — use `_base_dir()` (freeze-safe). Grep for backslash paths before committing.
- Sound: `winsound/WAV` only. If you add sounds, bundle them via `--add-data` and update `.bat`, `Kitty.spec`, AND `PyInstaller` args (3 places).
- `pet.post_ui` is the ONLY legal way to touch Tk from non-main threads.
- `set_duck` appears twice in `pet_window.py` (a short enable-only copy then the parameterized one). The later definition wins.

19. The Master Prompt (Paste-Ready)

Copy the text below into an AI tool when you sit down to work on this project again. It restores full context in one shot:

```
You are working on "Madhu", a personal Windows taskbar kitten + private journal app
written in Python (Tkinter + Pillow + pystray), for the user "Gayathree". Read
D:\KITTY\MASTER.md in FULL first — treat it as the authoritative spec. The repo is
https://github.com/ramsathwik2/something-something-/ (branch main). Important context to
honor: • The pet is 56×56 px living on the taskbar. src/app.py is the entry point
(single-instance Win32 mutex, ActivityMonitor, pystray tray, mainloop). src/pet_window.py
is the core class PetWindow (UI, behaviors, journal, bubbles, sounds, milestones).
src/animater.py, taskbar.py, activity.py, memory.py support it. • Runtime data: memory in
kitten_memory.json; the journal (Gayathree's Journal) in journal.json + journal_photos/,
optionally encrypted (journal.enc FERNET + scrypt, min 6-char password, 5-try throttle,
.enc.bak fallback, atomic temp+replace writes + .bak backups). Dev stores these in
D:\KITTY\assets\; frozen builds use %APPDATA%\TaskbarKitten\ (a one-time migration folds
dev data in). • Windows-only, winsound for sound (assets/meow.wav), Pillow sprites under
assets/sprites/, DPI is hardcoded so accept minor layout quirks. • Thread rule: Tk is
main-thread-only; cross-thread calls MUST use pet.post_ui(...) (drained every 300 ms). •
Menu indices count separators (entryconfig(4)=Pet name, entryconfig(9)=Mute, index
8=Don't sleep). Tray labels are callables over pet.tray_state. • NEVER commit things
other than what I explicitly stage; never create new doc files without asking; never
force-push; keep the app's warm, small-hearted voice; keep all messaging consistent with
the personality in MASTER.md. • Verifications: python -m py_compile on all src files,
then python _test_journal.py (16 checks), python _test_lock.py (11), test_walkfix.py
(13), test_dontsleep.py (23). Before any release also do the frozen-app smoke checklist
in MASTER.md section 15. • Build/release: update build_installer.bat, Kitty.spec and
PyInstaller --add-data flags together; ISCC installer\kitty.iss → KittySetup.exe; commit
+ push; the installer is distributed from the raw GitHub link. Begin by confirming you
have read MASTER.md and list what you understand the task to be.
```

Madhu — A personal project built with affection • Documented with care • September 19, 2026